Introduction to Optimization

Spring 2026

University of Science and Technology of China

Lecturer: Nick | Homework 4

Posted: Apr. 13th, 2025 | Due: May. 2nd, 2025

Name: Jinghao Chen | ID: PB23010359

Exercise 1:

Let f be a differentiable function with $\operatorname{dom} f = \mathbb{R}^n$.

(a) If ∇f is Lipschitz continuous with parameter $L > 0$, show that

$$g(x) := \frac{L}{2}\|x\|_2^2 - f(x)$$

is convex. In addition, if $x^\star$ is a minimizer of f, show that

$$\frac{1}{2L}\|\nabla f(x)\|_2^2 \leq f(x) - p^\star \leq \frac{L}{2}\|x - x^\star\|_2^2, \qquad \forall x.$$

(b) If f is strongly convex with parameter $m > 0$, show that if there is a minimizer $x^\star$, it is unique and it holds that

$$\frac{m}{2}\|x - x^\star\|_2^2 \leq f(x) - p^\star \leq \frac{1}{2m}\|\nabla f(x)\|_2^2, \qquad \forall x.$$

(c) Explain the implications of (a), (b) for optimization algorithms.

(d) If f satisfies the assumptions of both (a) and (b), show that

$$(\nabla f(y) - \nabla f(x))^T(y - x) \geq \frac{mL}{m+L}\|y - x\|_2^2 + \frac{1}{m+L}\|\nabla f(y) - \nabla f(x)\|_2^2, \qquad \forall x, y.$$

[Hint: The function

$$g := f - \frac{m}{2}\|\cdot\|_2^2$$

is convex and ∇g is Lipschitz continuous with parameter $L - m$. This implies that

$$(\nabla g(y) - \nabla g(x))^T(y - x) \geq \frac{1}{L-m}\|\nabla g(y) - \nabla g(x)\|_2^2, \qquad \forall x, y.]$$

Solution 1:
Let $p^\star = f(x^\star)$.

(a) Since ∇f is L-Lipschitz, we have the standard quadratic upper bound

$$f(y) \le f(x) + \nabla f(x)^T(y-x) + \frac{L}{2}\|y-x\|_2^2, \qquad \forall x, y.$$

Define

$$g(x) = \frac{L}{2}\|x\|_2^2 - f(x).$$

Then

$$\nabla g(x) = Lx - \nabla f(x),$$

and

$$\begin{aligned} g(y) - g(x) - \nabla g(x)^T(y-x) &= \frac{L}{2}\|y\|_2^2 - f(y) - \frac{L}{2}\|x\|_2^2 + f(x) - (Lx - \nabla f(x))^T(y-x) \\ &= \frac{L}{2}\|y-x\|_2^2 - \Big(f(y) - f(x) - \nabla f(x)^T(y-x)\Big) \ge 0. \end{aligned}$$

Hence g is convex.

Now let $x^\star$ be a minimizer of f. Since f is differentiable on $\mathbb{R}^n$,

$$\nabla f(x^\star) = 0.$$

Using the quadratic upper bound with base point $x^\star$,

$$f(x) \le f(x^\star) + \nabla f(x^\star)^T(x-x^\star) + \frac{L}{2}\|x-x^\star\|_2^2 = p^\star + \frac{L}{2}\|x-x^\star\|_2^2,$$

so

$$f(x) - p^\star \le \frac{L}{2}\|x-x^\star\|_2^2.$$

For the other inequality, choose

$$y = x - \frac{1}{L}\nabla f(x).$$

Then

$$\begin{aligned} f(y) &\le f(x) + \nabla f(x)^T\left(-\frac{1}{L}\nabla f(x)\right) + \frac{L}{2}\left\|-\frac{1}{L}\nabla f(x)\right\|_2^2 \\ &= f(x) - \frac{1}{L}\|\nabla f(x)\|_2^2 + \frac{1}{2L}\|\nabla f(x)\|_2^2 \\ &= f(x) - \frac{1}{2L}\|\nabla f(x)\|_2^2. \end{aligned}$$

Since $p^\star \le f(y)$,

$$\frac{1}{2L}\|\nabla f(x)\|_2^2 \le f(x) - p^\star.$$

Therefore,

$$\frac{1}{2L}\|\nabla f(x)\|_2^2 \le f(x) - p^\star \le \frac{L}{2}\|x - x^\star\|_2^2, \qquad \forall x.$$

(b) Since f is strongly convex with parameter $m > 0$,

$$f(y) \ge f(x) + \nabla f(x)^T(y - x) + \frac{m}{2}\|y - x\|_2^2, \qquad \forall x, y.$$

We first show uniqueness of the minimizer:

$\forall\ x,\ y \in \mathbb{R}^n,\ \theta \in (0,1)$, let $z = \theta x + (1-\theta)y$. By strong convexity,

$$\begin{cases} f(x) \ge f(z) + \nabla f(z)^T(x - z) + \dfrac{m}{2}\|x - z\|_2^2, \\ f(y) \ge f(z) + \nabla f(z)^T(y - z) + \dfrac{m}{2}\|y - z\|_2^2, \end{cases}$$

$$\begin{aligned} \Longrightarrow\ & \theta f(x) + (1-\theta) f(y) \\ \ge & f(z) + \nabla f(z)^T(\theta(x - z) + (1-\theta)(y - z)) + \frac{m}{2}\left(\theta\|x - z\|_2^2 + (1-\theta)\|y - z\|_2^2\right) \\ = & f(z) + \frac{m}{2}\theta(1-\theta)\|x - y\|_2^2. \end{aligned}$$

$$\xRightarrow{m>0} \theta f(x) + (1-\theta) f(y) > f(\theta x + (1-\theta)y)$$

Hence f is strictly convex, and the minimizer is unique.

Since $x^\star$ is a minimizer and f is differentiable,

$$\nabla f(x^\star) = 0.$$

Applying strong convexity with $x = x^\star$ gives

$$f(x) \ge f(x^\star) + \nabla f(x^\star)^T(x - x^\star) + \frac{m}{2}\|x - x^\star\|_2^2 = p^\star + \frac{m}{2}\|x - x^\star\|_2^2,$$

hence

$$\frac{m}{2}\|x - x^\star\|_2^2 \le f(x) - p^\star.$$

For the upper bound, from strong convexity we have for every y,

$$f(y) \ge f(x) + \nabla f(x)^T(y - x) + \frac{m}{2}\|y - x\|_2^2.$$

Taking the infimum over y yields

$$p^\star \ge \inf_y \left\{ f(x) + \nabla f(x)^T(y - x) + \frac{m}{2}\|y - x\|_2^2 \right\}.$$

The right-hand side is minimized at

$$y = x - \frac{1}{m}\nabla f(x),$$

and the minimum value equals

$$f(x) - \frac{1}{2m}\|\nabla f(x)\|_2^2.$$

Therefore

$$p^\star \geq f(x) - \frac{1}{2m}\|\nabla f(x)\|_2^2,$$

i.e.

$$f(x) - p^\star \leq \frac{1}{2m}\|\nabla f(x)\|_2^2.$$

Hence

$$\frac{m}{2}\|x - x^\star\|_2^2 \leq f(x) - p^\star \leq \frac{1}{2m}\|\nabla f(x)\|_2^2, \qquad \forall x.$$

(c) The inequalities in (a) and (b) provide error bounds relating three quantities:

$$f(x) - p^\star, \qquad \|x - x^\star\|_2, \qquad \|\nabla f(x)\|_2.$$

In particular:

- If f is L-smooth, then small gradient implies small suboptimality:

  $$\frac{1}{2L}\|\nabla f(x)\|_2^2 \leq f(x) - p^\star.$$

  Also, closeness to $x^\star$ implies small objective error:

  $$f(x) - p^\star \leq \frac{L}{2}\|x - x^\star\|_2^2.$$

- If f is m-strongly convex, then objective error controls distance to the optimizer:

  $$\frac{m}{2}\|x - x^\star\|_2^2 \leq f(x) - p^\star,$$

  and gradient controls objective error:

  $$f(x) - p^\star \leq \frac{1}{2m}\|\nabla f(x)\|_2^2.$$

Thus, in the strongly convex case, function error, gradient norm, and distance to the optimizer are all equivalent up to constants. This explains why gradient-based stopping criteria are meaningful, and why first-order methods behave much better in the strongly convex case.

If both assumptions hold, then combining (a) and (b) gives

$$2m\big(f(x)-p^\star\big) \le \|\nabla f(x)\|_2^2 \le 2L\big(f(x)-p^\star\big),$$

which is a fundamental basis for linear convergence results of gradient descent. The convergence speed depends on the condition number $\kappa = L/m$.

(d) Define

$$h(x) := f(x) - \frac{m}{2}\|x\|_2^2.$$

By strong convexity of f, h is convex. By the hint, ∇h is Lipschitz continuous with parameter $L-m$, and

$$(\nabla h(y) - \nabla h(x))^T(y-x) \ge \frac{1}{L-m}\|\nabla h(y) - \nabla h(x)\|_2^2.$$

Let

$$\Delta x := y - x, \qquad \Delta\nabla := \nabla f(y) - \nabla f(x).$$

Since

$$\nabla h(y) - \nabla h(x) = \Delta\nabla - m\Delta x,$$

the above inequality becomes

$$(\Delta\nabla - m\Delta x)^T\Delta x \ge \frac{1}{L-m}\|\Delta\nabla - m\Delta x\|_2^2.$$

Expanding both sides,

$$\Delta\nabla^T\Delta x - m\|\Delta x\|_2^2 \ge \frac{1}{L-m}\left(\|\Delta\nabla\|_2^2 - 2m\,\Delta\nabla^T\Delta x + m^2\|\Delta x\|_2^2\right).$$

Multiply by $L-m$:

$$(L-m)\Delta\nabla^T\Delta x - m(L-m)\|\Delta x\|_2^2 \ge \|\Delta\nabla\|_2^2 - 2m\,\Delta\nabla^T\Delta x + m^2\|\Delta x\|_2^2.$$

Rearranging,

$$(L+m)\Delta\nabla^T\Delta x \ge \|\Delta\nabla\|_2^2 + mL\|\Delta x\|_2^2.$$

Therefore

$$\Delta\nabla^T\Delta x \ge \frac{mL}{m+L}\|\Delta x\|_2^2 + \frac{1}{m+L}\|\Delta\nabla\|_2^2.$$

Replacing $\Delta x, \Delta\nabla$ by their definitions,

$$(\nabla f(y) - \nabla f(x))^T(y-x) \ge \frac{mL}{m+L}\|y-x\|_2^2 + \frac{1}{m+L}\|\nabla f(y) - \nabla f(x)\|_2^2.$$

■

Exercise 2:

Analyze the gradient descent method for each of the following cases.

(a) f has L-Lipschitz gradient. Show that

$$f(x^{(k)}) - f(x^{(k-1)}) \leq -t^{(k)} \left(1 - \frac{Lt^{(k)}}{2}\right) \|\nabla f(x^{(k-1)})\|_2^2,$$

therefore for $t^{(k)} \in \left(0, \frac{2}{L}\right)$ GD is a descent method.

[Hint: Use the quadratic upper bound (Hw1, problem 6a):

Let f be a differentiable function with $\operatorname{dom} f = \mathbb{R}^n$.

(a) If there exists $L \geq 0$ such that

$$\|\nabla f(y) - \nabla f(x)\|_2 \leq L\|y - x\|_2, \qquad \forall x, y,$$

show that

$$f(y) \leq f(x) + \nabla f(x)^T(y - x) + \frac{L}{2}\|y - x\|_2^2, \qquad \forall x, y.$$

[Hint: Use that $f(y) = f(x) + \int_0^1 \nabla f\big(x + t(y - x)\big)^T (y - x)\, dt$.]]

(b) In addition to the assumption in (a), f is lower-bounded. For fixed step-size $t = \frac{1}{L}$, show that

$$\min_{i=0,\ldots,k} \|\nabla f(x^{(i)})\|_2^2 \leq \frac{2L}{k+1}\big(f(x^{(0)}) - p^\star\big).$$

(c) In addition to the assumptions in (b), f is convex and attains the minimum at $x^\star$. Show that

$$f(x^{(k)}) - p^\star \leq \frac{L}{2k}\|x^{(0)} - x^\star\|_2^2.$$

(d) In addition to the assumptions in (c), f is strongly convex with parameter $m > 0$. Show that for fixed step-size

$$t \in \left(0, \frac{2}{m + L}\right)$$

it holds that

$$\|x^{(k)} - x^\star\|_2^2 \leq \rho^k \|x^{(0)} - x^\star\|_2^2, \qquad \rho := \left(1 - t\frac{2mL}{m + L}\right).$$

[Hint: Use the bound from problem 1d]

For the value of t that minimizes the upper bound, express ρ as a function of the condition number

$$\kappa := \frac{L}{m},$$

and also derive a bound for $f(x^{(k)}) - p^\star$ as a function of $f(x^{(0)}) - p^\star$.

(e) Explain the implications of parts (b) (for non-convex optimization) and parts (c), (d) for convex optimization by analyzing the number of iterations needed to attain a given accuracy ϵ.

Solution 2:
Let the GD iteration be

$$x^{(k)} = x^{(k-1)} - t^{(k)}\nabla f(x^{(k-1)}).$$

(a) Since ∇f is L-Lipschitz, we have the quadratic upper bound

$$f(y) \le f(x) + \nabla f(x)^T(y-x) + \frac{L}{2}\|y-x\|_2^2, \qquad \forall x, y.$$

Apply this with

$$x = x^{(k-1)}, \qquad y = x^{(k)} = x^{(k-1)} - t^{(k)}\nabla f(x^{(k-1)}).$$

Then

$$y - x = -t^{(k)}\nabla f(x^{(k-1)}),$$

hence

$$\begin{aligned} f(x^{(k)}) &\le f(x^{(k-1)}) + \nabla f(x^{(k-1)})^T\left(-t^{(k)}\nabla f(x^{(k-1)})\right) + \frac{L}{2}\left\|-t^{(k)}\nabla f(x^{(k-1)})\right\|_2^2 \\ &= f(x^{(k-1)}) - t^{(k)}\|\nabla f(x^{(k-1)})\|_2^2 + \frac{L(t^{(k)})^2}{2}\|\nabla f(x^{(k-1)})\|_2^2. \end{aligned}$$

Therefore

$$f(x^{(k)}) - f(x^{(k-1)}) \le -t^{(k)}\left(1 - \frac{Lt^{(k)}}{2}\right)\|\nabla f(x^{(k-1)})\|_2^2.$$

If

$$t^{(k)} \in \left(0, \frac{2}{L}\right),$$

then $1 - \frac{Lt^{(k)}}{2} > 0$, so GD is a descent method.

(b) Now fix $t = \frac{1}{L}$. Then from part (a),

$$f(x^{(k+1)}) - f(x^{(k)}) \le -\frac{1}{L}\left(1 - \frac{1}{2}\right)\|\nabla f(x^{(k)})\|_2^2 = -\frac{1}{2L}\|\nabla f(x^{(k)})\|_2^2.$$

Hence

$$\frac{1}{2L}\|\nabla f(x^{(k)})\|_2^2 \le f(x^{(k)}) - f(x^{(k+1)}).$$

Summing from $i = 0$ to k gives

$$\frac{1}{2L}\sum_{i=0}^{k}\|\nabla f(x^{(i)})\|_2^2 \le f(x^{(0)}) - f(x^{(k+1)}).$$

Since f is lower bounded and $p^\star = \inf f$,

$$f(x^{(k+1)}) \ge p^\star,$$

so

$$\sum_{i=0}^{k} \|\nabla f(x^{(i)})\|_2^2 \le 2L\big(f(x^{(0)}) - p^\star\big).$$

Therefore

$$\min_{i=0,\dots,k} \|\nabla f(x^{(i)})\|_2^2 \le \frac{1}{k+1}\sum_{i=0}^{k} \|\nabla f(x^{(i)})\|_2^2 \le \frac{2L}{k+1}\big(f(x^{(0)}) - p^\star\big).$$

(c) Assume in addition that f is convex and attains its minimum at $x^\star$. We still use the fixed step-size $t = \frac{1}{L}$.

First, from

$$x^{(k+1)} = x^{(k)} - \frac{1}{L}\nabla f(x^{(k)}),$$

we have

$$\begin{aligned}\|x^{(k+1)} - x^\star\|_2^2 &= \left\|x^{(k)} - x^\star - \frac{1}{L}\nabla f(x^{(k)})\right\|_2^2 \\ &= \|x^{(k)} - x^\star\|_2^2 - \frac{2}{L}\nabla f(x^{(k)})^T(x^{(k)} - x^\star) + \frac{1}{L^2}\|\nabla f(x^{(k)})\|_2^2.\end{aligned}$$

Rearranging,

$$\nabla f(x^{(k)})^T(x^{(k)} - x^\star) = \frac{L}{2}\Big(\|x^{(k)} - x^\star\|_2^2 - \|x^{(k+1)} - x^\star\|_2^2\Big) + \frac{1}{2L}\|\nabla f(x^{(k)})\|_2^2.$$

By convexity,

$$f(x^{(k)}) - p^\star = f(x^{(k)}) - f(x^\star) \le \nabla f(x^{(k)})^T(x^{(k)} - x^\star).$$

Hence

$$f(x^{(k)}) - p^\star \le \frac{L}{2}\Big(\|x^{(k)} - x^\star\|_2^2 - \|x^{(k+1)} - x^\star\|_2^2\Big) + \frac{1}{2L}\|\nabla f(x^{(k)})\|_2^2.$$

On the other hand, part (b) yields

$$f(x^{(k+1)}) \le f(x^{(k)}) - \frac{1}{2L}\|\nabla f(x^{(k)})\|_2^2.$$

Combining these two inequalities,

$$f(x^{(k+1)}) - p^\star \le \frac{L}{2}\Big(\|x^{(k)} - x^\star\|_2^2 - \|x^{(k+1)} - x^\star\|_2^2\Big).$$

Summing from $i = 0$ to $k-1$,

$$\sum_{i=0}^{k-1}\big(f(x^{(i+1)}) - p^\star\big) \le \frac{L}{2}\|x^{(0)} - x^\star\|_2^2.$$

Since GD is a descent method, the sequence $f(x^{(k)})$ is nonincreasing, so

$$f(x^{(k)}) - p^\star \le f(x^{(i)}) - p^\star, \qquad i = 1, \dots, k.$$

Therefore

$$k\big(f(x^{(k)}) - p^\star\big) \le \sum_{i=0}^{k-1} \big(f(x^{(i+1)}) - p^\star\big) \le \frac{L}{2}\|x^{(0)} - x^\star\|_2^2,$$

i.e.

$$f(x^{(k)}) - p^\star \le \frac{L}{2k}\|x^{(0)} - x^\star\|_2^2.$$

(d) Assume in addition that f is strongly convex with parameter $m > 0$. Let t be fixed. From Problem 1(d), for all x, y,

$$(\nabla f(y) - \nabla f(x))^T (y - x) \ge \frac{mL}{m+L}\|y - x\|_2^2 + \frac{1}{m+L}\|\nabla f(y) - \nabla f(x)\|_2^2.$$

Set $x = x^\star$, $y = x^{(k)}$. Since $\nabla f(x^\star) = 0$,

$$\nabla f(x^{(k)})^T (x^{(k)} - x^\star) \ge \frac{mL}{m+L}\|x^{(k)} - x^\star\|_2^2 + \frac{1}{m+L}\|\nabla f(x^{(k)})\|_2^2.$$

Now

$$\begin{aligned}\|x^{(k+1)} - x^\star\|_2^2 &= \|x^{(k)} - x^\star - t\nabla f(x^{(k)})\|_2^2 \\ &= \|x^{(k)} - x^\star\|_2^2 - 2t\,\nabla f(x^{(k)})^T (x^{(k)} - x^\star) + t^2\|\nabla f(x^{(k)})\|_2^2.\end{aligned}$$

Using the previous lower bound,

$$\begin{aligned}\|x^{(k+1)} - x^\star\|_2^2 &\le \|x^{(k)} - x^\star\|_2^2 - 2t\left[\frac{mL}{m+L}\|x^{(k)} - x^\star\|_2^2 + \frac{1}{m+L}\|\nabla f(x^{(k)})\|_2^2\right] \\ &\quad + t^2\|\nabla f(x^{(k)})\|_2^2 \\ &= \left(1 - t\frac{2mL}{m+L}\right)\|x^{(k)} - x^\star\|_2^2 + \left(t^2 - \frac{2t}{m+L}\right)\|\nabla f(x^{(k)})\|_2^2.\end{aligned}$$

If

$$t \in \left(0, \frac{2}{m+L}\right),$$

then $t^2 - \frac{2t}{m+L} < 0$, hence

$$\|x^{(k+1)} - x^\star\|_2^2 \le \rho\,\|x^{(k)} - x^\star\|_2^2, \qquad \rho := 1 - t\frac{2mL}{m+L}.$$

By recursion,

$$\|x^{(k)} - x^\star\|_2^2 \le \rho^k \|x^{(0)} - x^\star\|_2^2.$$

Since ρ decreases with t, the best choice is the endpoint

$$t_\star = \frac{2}{m+L},$$

which gives

$$\rho_\star = 1 - \frac{4mL}{(m+L)^2} = \frac{(L-m)^2}{(L+m)^2}.$$

Writing this in terms of the condition number $\kappa = \frac{L}{m}$,

$$\rho_\star = \left(\frac{\kappa-1}{\kappa+1}\right)^2.$$

Finally, using

$$f(x) - p^\star \le \frac{L}{2}\|x - x^\star\|_2^2 \qquad \text{and} \qquad f(x) - p^\star \ge \frac{m}{2}\|x - x^\star\|_2^2,$$

we obtain

$$\begin{aligned} f(x^{(k)}) - p^\star &\le \frac{L}{2}\|x^{(k)} - x^\star\|_2^2 \\ &\le \frac{L}{2}\rho^k\|x^{(0)} - x^\star\|_2^2 \\ &\le \frac{L}{m}\rho^k\big(f(x^{(0)}) - p^\star\big). \end{aligned}$$

Hence

$$f(x^{(k)}) - p^\star \le \kappa\,\rho^k\big(f(x^{(0)}) - p^\star\big).$$

In particular, for $t = t_\star$,

$$f(x^{(k)}) - p^\star \le \kappa\left(\frac{\kappa-1}{\kappa+1}\right)^{2k}\big(f(x^{(0)}) - p^\star\big).$$

(e) The implications are as follows.

For the non-convex case in part (b), the guarantee is on stationarity:

$$\min_{i=0,\dots,k}\|\nabla f(x^{(i)})\|_2^2 \le \frac{2L}{k+1}\big(f(x^{(0)}) - p^\star\big).$$

Therefore, to ensure

$$\min_{i=0,\dots,k}\|\nabla f(x^{(i)})\|_2 \le \epsilon,$$

it suffices that

$$k+1 \ge \frac{2L\big(f(x^{(0)}) - p^\star\big)}{\epsilon^2}.$$

Thus the iteration complexity is

$$O\left(\frac{1}{\epsilon^2}\right)$$

for finding an ϵ-stationary point.

For the convex case in part (c),

$$f(x^{(k)}) - p^\star \leq \frac{L}{2k}\|x^{(0)} - x^\star\|_2^2.$$

To ensure

$$f(x^{(k)}) - p^\star \leq \epsilon,$$

it suffices that

$$k \geq \frac{L}{2\epsilon}\|x^{(0)} - x^\star\|_2^2.$$

Hence the iteration complexity is

$$O\left(\frac{1}{\epsilon}\right),$$

i.e. sublinear convergence.

For the strongly convex case in part (d),

$$\|x^{(k)} - x^\star\|_2^2 \leq \rho^k\|x^{(0)} - x^\star\|_2^2, \qquad 0 < \rho < 1.$$

Thus to ensure

$$\|x^{(k)} - x^\star\|_2^2 \leq \epsilon,$$

it suffices that

$$k \geq \frac{\log(\|x^{(0)} - x^\star\|_2^2/\epsilon)}{-\log\rho}.$$

This is

$$O\left(\log\frac{1}{\epsilon}\right),$$

i.e. linear (geometric) convergence. Using $t_\star = \frac{2}{m+L}$, we have

$$\rho_\star = \left(\frac{\kappa - 1}{\kappa + 1}\right)^2,$$

so the dependence on problem conditioning is captured by κ.

■

Exercise 3:

True or False? Briefly explain the reason.

(a) In descent methods, the particular choice of search direction does not matter so much.

(b) In descent methods, the particular choice of line search does not matter so much.

(c) When the gradient method is started from a point near the solution, it will converge very quickly.

(d) When Newton's method is started from a point near the solution, it will converge very quickly.

(e) Newton's method with step size $t^{(k)} \equiv 1$ always works; damping (i.e., using $t^{(k)} < 1$) is used only to improve the speed of convergence.

(f) Using the gradient method to minimize $f(Ty)$, where $Ty = x$ and T is non-singular, can greatly improve the convergence speed when T is chosen appropriately.

(g) Using Newton's method to minimize $f(Ty)$, where $Ty = x$ and T is non-singular, can greatly improve the convergence speed when T is chosen appropriately.

Solution 3:

(a) **False.** The search direction matters a lot. Different descent directions can lead to very different convergence behavior and speed; some may be much more effective than others.

(b) **False.** The line search is also important. Poor step sizes can make the method extremely slow or even unstable, while a good line search can ensure descent and improve convergence.

(c) **False.** Gradient descent generally has only linear convergence, and its speed depends strongly on the conditioning of the problem. Even near the solution, it may still converge slowly.

(d) **True.** Under standard assumptions, Newton's method has local quadratic convergence, so if it is started sufficiently close to the solution, it converges very quickly.

(e) **False.** Newton's method with full step $t^{(k)} \equiv 1$ may fail when started far from the solution. Damping is mainly used to improve robustness and global convergence, not just speed.

(f) **True.** A suitable nonsingular transformation $x = Ty$ can significantly improve the conditioning of the problem. Since gradient descent is not affine invariant, this can greatly accelerate convergence.

(g) **False.** Newton's method is affine invariant. If applied to $f(Ty)$, the iterates in the x-space correspond exactly to those of Newton's method applied directly to $f(x)$, so an appropriate T does not fundamentally improve its convergence behavior.

■

Exercise 4:

Develop code for gradient descent and Newton's method and use it to solve the following Machine Learning problems. You may use real datasets from the UCI Machine Learning Repository.

- **Linear regression:** Given N examples $\{(a_i, b_i)\}_{i=1}^N$ ($a_i \in \mathbb{R}^d$, $b_i \in \mathbb{R}$), the goal is to build a linear prediction model so as to

$$\textbf{minimize} \quad f(x) := \sum_{i=1}^{N} \|a_i^T x - b_i\|_2^2.$$

- **Logistic regression:** Given N examples $\{(a_i, y_i)\}_{i=1}^N$ ($a_i \in \mathbb{R}^{d+1}$ is the feature and $y_i \in \{0, 1\}$ is the corresponding label), the goal is to build a binary classifier so that $\text{sign}(a_i^T x)$ predicts the label y_i. This can be accomplished by aiming to

$$\textbf{minimize} \quad f(x) := \sum_{i=1}^{N} \left\{ \log\left(1 + e^{a_i^T x}\right) - (1 - y_i) a_i^T x \right\}.$$

  [Given a feature $\bar{a}_i \in \mathbb{R}^d$ in UCI, set $a_i := (\bar{a}_i, 1)$ (augmented feature); this is done so that $\mathbb{R}^d$ is separated in two halfspaces, each corresponding to a class.]

(a) For each problem analyze the properties of the objective (gradient Lipschitz continuity and strong convexity). Estimate the values L, m from the chosen datasets. [For any ill-conditioned instances you may augment the objective with the term $\frac{1}{2}\gamma\|x\|_2^2$, $\gamma > 0$; explain how you chose γ.]

(b) For each problem, use CVXPY to compute the optimal solution and value $(x^\star, p^\star)$.

(c) For each problem and both methods generate two figures, by plotting

  (i) $\|x^{(k)} - x^\star\|_2$
  (ii) $f(x^{(k)}) - p^\star$

  vs. iteration count k. [Use logarithmic scale.]

(d) For each problem, report the run-time required for each method to reach an RMSE (rooted-mean squared error)

$$\sqrt{\frac{1}{d}\|x^{(k)} - x^\star\|_2^2} \leq \epsilon, \qquad \text{for } \epsilon \in \{10^{-2}, 10^{-3}, 10^{-4}\}.$$

(e) Explain your results in view of the theoretical analysis.

Solution 4:
We implemented gradient descent and Newton's method for both linear regression and logistic regression, and used CVXPY to compute a reference optimizer $x^\star$ and optimal value $p^\star$.

In the experiments, we used the diabetes dataset for linear regression and the breast cancer dataset for logistic regression. All features were standardized, and then a column of ones was appended to incorporate the intercept term. Hence the final dimensions are $d = 11$ for linear regression and $d = 31$ for logistic regression.

(a) **Gradient Lipschitz continuity and strong convexity.**

Linear regression. The objective is

$$f(x) = \|Ax - b\|_2^2.$$

Its gradient and Hessian are

$$\nabla f(x) = 2A^T(Ax - b), \qquad \nabla^2 f(x) = 2A^T A.$$

Therefore the gradient is Lipschitz continuous with constant

$$L = \lambda_{\max}(2A^T A),$$

and, since the data matrix has full column rank after preprocessing, the objective is strongly convex with parameter

$$m = \lambda_{\min}(2A^T A).$$

From the dataset, we estimated

$$L = 3557.402303, \qquad m = 7.567685,$$

so the condition number is

$$\kappa = \frac{L}{m} = 470.077999.$$

No additional regularization was needed, so $\gamma = 0$.

Logistic regression. We optimized the regularized objective

$$f(x) = \sum_{i=1}^{N} \Big(\log(1 + e^{a_i^T x}) - (1 - y_i) a_i^T x \Big) + \frac{\gamma}{2} \|x\|_2^2,$$

with $\gamma = 1$. Let

$$\sigma(t) = \frac{1}{1 + e^{-t}}.$$

Then

$$\nabla f(x) = A^T(\sigma(Ax) - (1 - y)) + \gamma x,$$

and

$$\nabla^2 f(x) = A^T W(x) A + \gamma I,$$

where

$$W(x) = \operatorname{diag}\big(\sigma(z_i)(1-\sigma(z_i))\big), \qquad z_i = a_i^T x.$$

Since

$$0 \le \sigma(z_i)(1-\sigma(z_i)) \le \frac{1}{4},$$

we have

$$\nabla^2 f(x) \preceq \frac{1}{4} A^T A + \gamma I.$$

Hence the gradient is Lipschitz continuous with

$$L = \frac{1}{4}\lambda_{\max}(A^T A) + \gamma,$$

and, because

$$\nabla^2 f(x) \succeq \gamma I,$$

the objective is strongly convex with parameter

$$m = \gamma = 1.$$

From the dataset, we estimated

$$L = 1890.308693, \qquad m = 1,$$

so

$$\kappa = \frac{L}{m} = 1890.308693.$$

(b) **CVXPY solution.**

For both problems, the reference optimizer $x^\star$ and optimal value $p^\star$ were computed by CVXPY and then used to evaluate

$$\|x^{(k)} - x^\star\|_2 \qquad \text{and} \qquad f(x^{(k)}) - p^\star.$$

The optimal values are:

$$p^\star_{\text{lin}} = 1.263986 \times 10^6, \qquad p^\star_{\text{log}} = 3.777823 \times 10^1.$$

(c) **Convergence plots.**

The required plots are shown below.

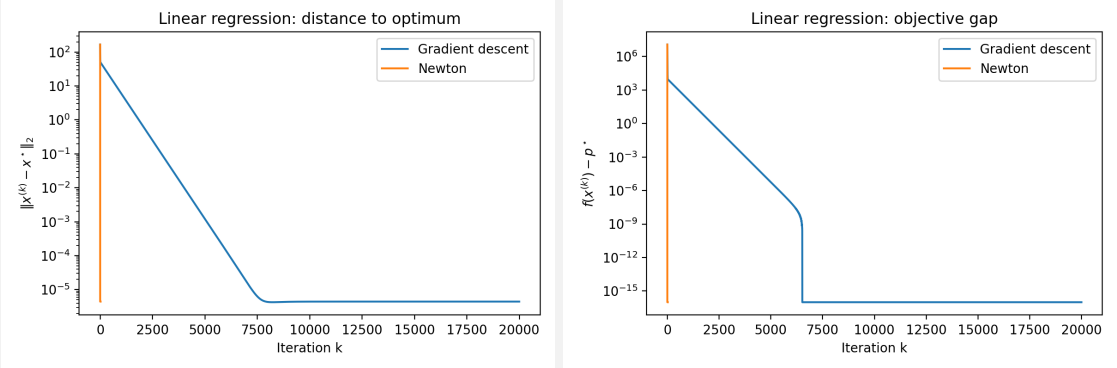

Figure 1: Linear regression: $\|x^{(k)} - x^\star\|_2$ and $f(x^{(k)}) - p^\star$ versus iteration count.

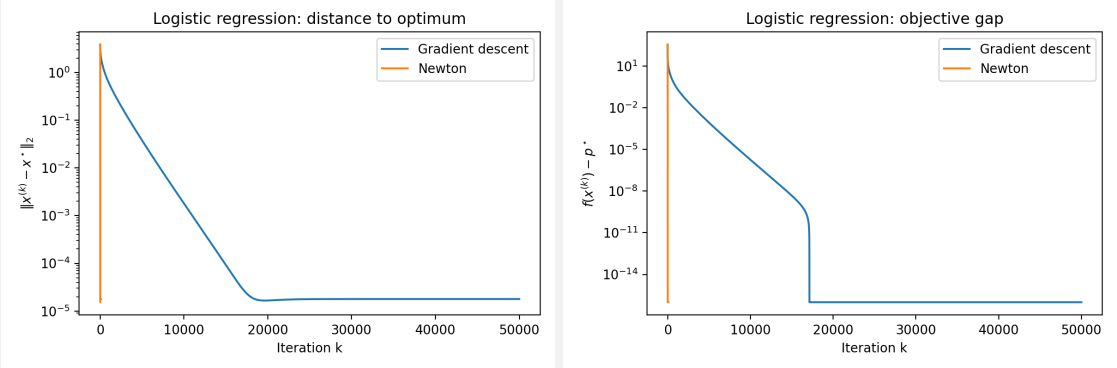

Figure 2: Logistic regression: $\|x^{(k)} - x^\star\|_2$ and $f(x^{(k)}) - p^\star$ versus iteration count.

From the plots, gradient descent exhibits linear convergence in both problems, while Newton's method converges dramatically faster. For linear regression, Newton's method essentially reaches the optimum in one step. For logistic regression, Newton's method converges within only a few iterations, while gradient descent requires several thousand iterations.

(d) **Run-time to reach the prescribed RMSE levels.**

We used the criterion

$$\sqrt{\frac{1}{d}\|x^{(k)} - x^\star\|_2^2} \le \epsilon, \qquad \epsilon \in \{10^{-2}, 10^{-3}, 10^{-4}\}.$$

Linear regression:

Method	$\epsilon = 10^{-2}$	$\epsilon = 10^{-3}$	$\epsilon = 10^{-4}$
Gradient descent	(3448, 0.072157 s)	(4529, 0.093280 s)	(5608, 0.113464 s)
Newton	(1, 0.000165 s)	(1, 0.000165 s)	(1, 0.000165 s)

Logistic regression:

Method	$\epsilon = 10^{-2}$	$\epsilon = 10^{-3}$	$\epsilon = 10^{-4}$
Gradient descent	(4395, 1.000440 s)	(8109, 1.827914 s)	(11981, 2.820052 s)
Newton	(7, 0.007921 s)	(7, 0.007921 s)	(8, 0.008460 s)

Each pair above is of the form (iterations, time).

(e) **Explanation of the results.**

The results are fully consistent with the theory.

For linear regression, the objective is a strongly convex quadratic function. Gradient descent with step size $1/L$ converges linearly:

$$\|x^{(k)} - x^\star\|_2 \le \left(1 - \frac{m}{L}\right)^k \|x^{(0)} - x^\star\|_2.$$

Here,

$$1 - \frac{m}{L} = 1 - \frac{7.567685}{3557.402303} \approx 0.99787.$$

Thus the convergence is linear but relatively slow compared with Newton's method. Since the Hessian is constant, Newton's method solves the quadratic problem essentially in one iteration, exactly as seen in the plots and timing table.

For logistic regression, after adding $\frac{1}{2}\gamma\|x\|_2^2$ with $\gamma = 1$, the objective becomes globally strongly convex. Gradient descent again converges linearly, but with a worse condition number:

$$\kappa \approx 1890.308693,$$

and rate

$$1 - \frac{m}{L} = 1 - \frac{1}{1890.308693} \approx 0.99947.$$

Since this number is much closer to 1, the method needs far more iterations than in linear regression. Newton's method, however, exploits second-order curvature information and therefore converges in only a few steps.

In summary, the experiments confirm the expected theoretical behavior: gradient descent is globally convergent but slow when the condition number is large, whereas Newton's method is much faster, especially for strongly convex problems and particularly for quadratic objectives.

Scripts:

```
import numpy as np
import pandas as pd
import time
from pathlib import Path
from numpy.linalg import norm, solve, eigvalsh
from scipy.optimize import minimize
from sklearn.datasets import load_diabetes, load_breast_cancer
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
```

```
OUTDIR = Path("results_opt_hw")
OUTDIR.mkdir(exist_ok=True)

# ============================================================
# Data
# ============================================================

# Linear regression dataset: diabetes (real regression dataset in
    scikit-learn)
lin = load_diabetes()
X_lin_raw, b_lin = lin.data, lin.target.astype(float)

# Standardize features and append a constant 1 to model an
    intercept
X_lin = StandardScaler().fit_transform(X_lin_raw)
A_lin = np.hstack([X_lin, np.ones((X_lin.shape[0], 1))])

# Logistic regression dataset: Breast Cancer Wisconsin (Diagnostic)
logd = load_breast_cancer()
X_log_raw = logd.data
y_log = logd.target.astype(float)  # 0/1 labels
X_log = StandardScaler().fit_transform(X_log_raw)
A_log = np.hstack([X_log, np.ones((X_log.shape[0], 1))])

n_lin, d_lin = A_lin.shape
n_log, d_log = A_log.shape

# Regularization:
#   linear: not needed, because A has full column rank after
    augmentation
#   logistic: added to guarantee global strong convexity
gamma_lin = 0.0
gamma_log = 1.0

# ============================================================
# Objectives
# ============================================================

def sigmoid(z):
    out = np.empty_like(z)
    pos = z >= 0
    out[pos] = 1.0 / (1.0 + np.exp(-z[pos]))
    ez = np.exp(z[~pos])
    out[~pos] = ez / (1.0 + ez)
    return out

def f_lin(x):
    r = A_lin @ x - b_lin
    return float(r @ r + 0.5 * gamma_lin * (x @ x))

def g_lin(x):
    r = A_lin @ x - b_lin
    return 2 * A_lin.T @ r + gamma_lin * x
```

```
def H_lin(_x):
    return 2 * A_lin.T @ A_lin + gamma_lin * np.eye(d_lin)

def f_log(x):
    z = A_log @ x
    return float(np.sum(np.logaddexp(0.0, z) - (1.0 - y_log) * z) +
        0.5 * gamma_log * (x @ x))

def g_log(x):
    z = A_log @ x
    s = sigmoid(z)
    return A_log.T @ (s - (1.0 - y_log)) + gamma_log * x

def H_log(x):
    z = A_log @ x
    s = sigmoid(z)
    w = s * (1.0 - s)
    return A_log.T @ (A_log * w[:, None]) + gamma_log * np.eye(
       d_log)

# ============================================================
# Lipschitz / strong convexity constants
# ============================================================

eig_lin = eigvalsh(2 * A_lin.T @ A_lin + gamma_lin * np.eye(d_lin))
L_lin = eig_lin[-1]
m_lin = eig_lin[0]

eig_log_AtA = eigvalsh(A_log.T @ A_log)
L_log = 0.25 * eig_log_AtA[-1] + gamma_log
m_log = gamma_log

# ============================================================
# CVXPY reference solution (if available)
# ============================================================

def solve_with_reference():
    try:
        import cvxpy as cp

        # linear
        x_lin = cp.Variable(d_lin)
        obj_lin = cp.sum_squares(A_lin @ x_lin - b_lin) + 0.5 *
           gamma_lin * cp.sum_squares(x_lin)
        prob_lin = cp.Problem(cp.Minimize(obj_lin))
        prob_lin.solve(solver=cp.SCS, verbose=False)
        x_star_lin = np.asarray(x_lin.value).reshape(-1)
        p_star_lin = float(prob_lin.value)

        # logistic
        x_log = cp.Variable(d_log)
        z = A_log @ x_log
```

```
        obj_log = cp.sum(cp.logistic(z) - cp.multiply(1.0 - y_log,
            z)) + 0.5 * gamma_log * cp.sum_squares(x_log)
        prob_log = cp.Problem(cp.Minimize(obj_log))
        prob_log.solve(solver=cp.SCS, verbose=False)
        x_star_log = np.asarray(x_log.value).reshape(-1)
        p_star_log = float(prob_log.value)

        print("Reference solver: CVXPY")
        return x_star_lin, p_star_lin, x_star_log, p_star_log

    except Exception as e:
        print("CVXPY unavailable, using analytic/high-precision
            fallback.")
        print("Reason:", e)

        # Linear regression has a closed-form solution
        x_star_lin = solve(H_lin(None), 2 * A_lin.T @ b_lin)
        p_star_lin = f_lin(x_star_lin)

        # High-precision trust-region solve for logistic
        res = minimize(
            f_log, np.zeros(d_log), jac=g_log, hess=H_log, method="
                trust-exact",
            options={"gtol": 1e-14, "maxiter": 200}
        )
        x_star_log = res.x
        p_star_log = f_log(x_star_log)
        return x_star_lin, p_star_lin, x_star_log, p_star_log

# ============================================================
# Algorithms
# ============================================================

def run_gd(f, g, x0, x_star, p_star, L, maxit=50000, tol_rmse=1e
    -12):
    x = x0.copy()
    xs = [x.copy()]
    fs = [f(x)]
    ts = [0.0]
    t0 = time.perf_counter()

    for _ in range(maxit):
        x = x - (1.0 / L) * g(x)
        xs.append(x.copy())
        fs.append(f(x))
        ts.append(time.perf_counter() - t0)
        rmse = norm(x - x_star) / np.sqrt(x_star.size)
        if rmse <= tol_rmse:
            break

    return {"xs": xs, "fs": np.array(fs), "ts": np.array(ts)}

def run_newton(f, g, H, x0, x_star, p_star, maxit=100, tol_rmse=1e
```

```
    -12, alpha=1e-4, beta=0.5):
    x = x0.copy()
    xs = [x.copy()]
    fs = [f(x)]
    ts = [0.0]
    t0 = time.perf_counter()

    for _ in range(maxit):
        grad = g(x)
        hess = H(x)
        d = solve(hess, -grad)
        fx = fs[-1]
        gd = grad @ d

        t = 1.0
        while f(x + t * d) > fx + alpha * t * gd:
            t *= beta
            if t < 1e-16:
                break

        x = x + t * d
        xs.append(x.copy())
        fs.append(f(x))
        ts.append(time.perf_counter() - t0)

        rmse = norm(x - x_star) / np.sqrt(x_star.size)
        if rmse <= tol_rmse:
            break

    return {"xs": xs, "fs": np.array(fs), "ts": np.array(ts)}

def threshold_stats(run, x_star, eps_list=(1e-2, 1e-3, 1e-4)):
    rows = []
    for eps in eps_list:
        for k, x in enumerate(run["xs"]):
            rmse = norm(x - x_star) / np.sqrt(x_star.size)
            if rmse <= eps:
                rows.append({
                    "epsilon": eps,
                    "iterations": int(k),
                    "time_sec": float(run["ts"][k]),
                    "rmse_at_hit": float(rmse),
                })
                break
    return pd.DataFrame(rows)

def metrics(run, x_star, p_star):
    dist = np.array([norm(x - x_star) for x in run["xs"]], dtype=
        float)
    gap = np.array(run["fs"] - p_star, dtype=float)
    return np.maximum(dist, 1e-16), np.maximum(gap, 1e-16)

def save_plots(problem_name, gd_run, nt_run, x_star, p_star):
```

```
    gd_dist, gd_gap = metrics(gd_run, x_star, p_star)
    nt_dist, nt_gap = metrics(nt_run, x_star, p_star)

    plt.figure(figsize=(6, 4))
    plt.plot(range(len(gd_dist)), gd_dist, label="Gradient descent"
        )
    plt.plot(range(len(nt_dist)), nt_dist, label="Newton")
    plt.yscale("log")
    plt.xlabel("Iteration k")
    plt.ylabel(r"$\|x^{(k)} - x^\star\|_2$")
    plt.title(f"{problem_name}: distance to optimum")
    plt.legend()
    plt.tight_layout()
    plt.savefig(OUTDIR / f"{problem_name.lower().replace(' ', '_')}
        _distance.png", dpi=200)
    plt.close()

    plt.figure(figsize=(6, 4))
    plt.plot(range(len(gd_gap)), gd_gap, label="Gradient descent")
    plt.plot(range(len(nt_gap)), nt_gap, label="Newton")
    plt.yscale("log")
    plt.xlabel("Iteration k")
    plt.ylabel(r"$f(x^{(k)}) - p^\star$")
    plt.title(f"{problem_name}: objective gap")
    plt.legend()
    plt.tight_layout()
    plt.savefig(OUTDIR / f"{problem_name.lower().replace(' ', '_')}
        _gap.png", dpi=200)
    plt.close()

def main():
    x_star_lin, p_star_lin, x_star_log, p_star_log =
        solve_with_reference()

    gd_lin = run_gd(f_lin, g_lin, np.zeros(d_lin), x_star_lin,
        p_star_lin, L_lin, maxit=20000)
    nt_lin = run_newton(f_lin, g_lin, H_lin, np.zeros(d_lin),
        x_star_lin, p_star_lin, maxit=20)

    gd_log = run_gd(f_log, g_log, np.zeros(d_log), x_star_log,
        p_star_log, L_log, maxit=50000)
    nt_log = run_newton(f_log, g_log, H_log, np.zeros(d_log),
        x_star_log, p_star_log, maxit=100)

    save_plots("Linear regression", gd_lin, nt_lin, x_star_lin,
        p_star_lin)
    save_plots("Logistic regression", gd_log, nt_log, x_star_log,
        p_star_log)

    summary = pd.DataFrame([
        {
            "problem": "Linear regression",
            "samples": n_lin,
```

```
            "dimension": d_lin,
            "gamma": gamma_lin,
            "L": float(L_lin),
            "m": float(m_lin),
            "condition_number_L_over_m": float(L_lin / m_lin),
            "p_star": float(p_star_lin),
        },
        {
            "problem": "Logistic regression",
            "samples": n_log,
            "dimension": d_log,
            "gamma": gamma_log,
            "L": float(L_log),
            "m": float(m_log),
            "condition_number_L_over_m": float(L_log / m_log),
            "p_star": float(p_star_log),
        },
    ])
    summary.to_csv(OUTDIR / "summary_table.csv", index=False)

    runtime = pd.concat([
        threshold_stats(gd_lin, x_star_lin).assign(problem_method="
            Linear-GD"),
        threshold_stats(nt_lin, x_star_lin).assign(problem_method="
            Linear-Newton"),
        threshold_stats(gd_log, x_star_log).assign(problem_method="
            Logistic-GD"),
        threshold_stats(nt_log, x_star_log).assign(problem_method="
            Logistic-Newton"),
    ], ignore_index=True)
    runtime.to_csv(OUTDIR / "runtime_table.csv", index=False)

    print("\nSummary")
    print(summary.to_string(index=False))

    print("\nRuntime to reach RMSE thresholds")
    print(runtime[["problem_method", "epsilon", "iterations", "
        time_sec", "rmse_at_hit"]].to_string(index=False))

if __name__ == "__main__":
    main()
```

■

Exercise 5:

Equality constrained entropy maximization. Consider the equality constrained entropy maximization problem

$$\begin{aligned} \textbf{minimize} \quad & f(x) = \sum_{i=1}^{n} x_i \log x_i \\ \text{subject to} \quad & Ax = b, \end{aligned}$$

with $\operatorname{dom} f = \mathbb{R}^n_{++}$ and $A \in \mathbb{R}^{p\times n}$, with $p < n$. (See Exercise 10.9 for some relevant analysis.)

Generate a problem instance with $n = 100$ and $p = 30$ by choosing A randomly (checking that it has full rank), choosing $\hat{x}$ as a random positive vector (e.g., with entries uniformly distributed on $[0, 1]$) and then setting $b = A\hat{x}$. (Thus, $\hat{x}$ is feasible.)

Compute the solution of the problem using the following method:

(a) *Standard Newton method.* You can use initial point $x^{(0)} = \hat{x}$.

Solution 5:
We consider

$$\begin{aligned} \textbf{minimize} \quad & f(x) = \sum_{i=1}^{n} x_i \log x_i \qquad \operatorname{dom} f = \mathbb{R}^n_{++}. \\ \text{subject to} \quad & Ax = b, \end{aligned}$$

For this problem,

$$\nabla f(x) = \log x + \mathbf{1}, \qquad \nabla^2 f(x) = \operatorname{diag}\left(\frac{1}{x_1}, \ldots, \frac{1}{x_n}\right).$$

Since $x_i > 0$, the Hessian is positive definite, so f is strictly convex.

Starting from a feasible point $x^{(0)} = \hat{x}$ with $A\hat{x} = b$, the equality constrained Newton step Δx is obtained from

$$\begin{aligned} \underset{\Delta x}{\textbf{minimize}} \quad & \nabla f(x)^T \Delta x + \frac{1}{2} \Delta x^T \nabla^2 f(x) \Delta x \\ \text{subject to} \quad & A\Delta x = 0. \end{aligned}$$

Its KKT system is

$$\begin{bmatrix} \nabla^2 f(x) & A^T \\ A & 0 \end{bmatrix} \begin{bmatrix} \Delta x \\ w \end{bmatrix} = \begin{bmatrix} -\nabla f(x) \\ 0 \end{bmatrix}.$$

Thus, for the entropy objective,

$$\begin{bmatrix} \operatorname{diag}(1/x) & A^T \\ A & 0 \end{bmatrix} \begin{bmatrix} \Delta x \\ w \end{bmatrix} = \begin{bmatrix} -(\log x + \mathbf{1}) \\ 0 \end{bmatrix}.$$

Using $\operatorname{diag}(1/x)^{-1} = \operatorname{diag}(x)$, we can write

$$\Delta x = -\operatorname{diag}(x)\big(\log x + \mathbf{1} + A^T w\big),$$

where w solves

$$A \operatorname{diag}(x) A^T w = -A \operatorname{diag}(x)(\log x + \mathbf{1}).$$

The update is

$$x^+ = x + t\Delta x,$$

where t is chosen by backtracking line search to ensure $x^+ \in \mathbb{R}^n_{++}$ and sufficient decrease. Since $A\Delta x = 0$, all iterates remain feasible:

$$A(x + t\Delta x) = Ax + tA\Delta x = b.$$

We generated a random instance with $n = 100$ and $p = 30$, choosing $A \in \mathbb{R}^{30\times 100}$ randomly and verifying that it has full rank, then choosing a random positive vector $\hat{x}$ and setting

$$b = A\hat{x}.$$

Hence $\hat{x}$ is feasible and was used as the initial point.

For the generated instance, the numerical results were:

$$\operatorname{rank}(A) = 30, \qquad \text{iterations} = 8,$$

$$f(x^{(0)}) = -26.818061118030787, \qquad f(x^\star) = -33.12710780279478.$$

The final feasibility residual and stationarity residual were

$$\|Ax^\star - b\|_2 = 2.7330716731741607 \times 10^{-14},$$

$$\|\nabla f(x^\star) + A^T\nu^\star\|_\infty = 7.257538459093382 \times 10^{-10}.$$

Thus the method converged to high accuracy.

The iteration history is:

k	$f(x^{(k)})$	$\lambda(x^{(k)})^2/2$	$\|Ax^{(k)} - b\|_2$
0	-26.8180611180	7.1245633754	0.000×10^{0}
1	-32.4582941311	$4.3215733648 \times 10^{-1}$	1.454×10^{-14}
2	-32.9507074296	$9.9299435643 \times 10^{-2}$	1.912×10^{-14}
3	-33.0780393631	$3.9056131947 \times 10^{-2}$	2.269×10^{-14}
4	-33.1235739220	$3.3735634962 \times 10^{-3}$	1.630×10^{-14}
5	-33.1270925679	$1.5192329576 \times 10^{-5}$	2.014×10^{-14}
6	-33.1271078025	$2.6608417102 \times 10^{-10}$	2.611×10^{-14}
7	-33.1271078028	$1.2905301912 \times 10^{-15}$	2.733×10^{-14}

Therefore, the standard Newton method solves this equality constrained entropy minimization problem very efficiently.

Scripts:

```
import numpy as np

np.set_printoptions(precision=6, suppress=True)

def f(x):
    return np.sum(x * np.log(x))

def grad(x):
    return np.log(x) + 1.0

def generate_instance(n=100, p=30, seed=20260415):
    rng = np.random.default_rng(seed)
    while True:
        A = rng.standard_normal((p, n))
        if np.linalg.matrix_rank(A) == p:
            break
    # strictly positive initial feasible point
    xhat = rng.uniform(0.1, 1.0, size=n)
    b = A @ xhat
    return A, b, xhat

def newton_eq_entropy(A, b, x0, alpha=0.01, beta=0.5, tol=1e-12,
   max_iter=100):
```

```
    x = x0.copy().astype(float)
    hist = []

    for k in range(max_iter):
        g = grad(x)
        Hinv = x  # since Hessian = diag(1/x), its inverse is diag(x)

        # Solve reduced KKT system:
        # (A diag(x) A^T) w = -A diag(x) g
        M = A @ (Hinv[:, None] * A.T)
        rhs = -A @ (Hinv * g)
        w = np.linalg.solve(M, rhs)

        # Newton step
        dx = -Hinv * (g + A.T @ w)

        # Newton decrement squared
        lam_sq = -g @ dx

        fval = f(x)
        feas = np.linalg.norm(A @ x - b)
        hist.append((k, fval, lam_sq / 2.0, feas))

        if lam_sq / 2.0 <= tol:
            return x, w, hist

        # backtracking line search
        t = 1.0

        # maintain positivity
        neg = dx < 0
        if np.any(neg):
            t = min(t, 0.99 * np.min(-x[neg] / dx[neg]))

        gd = g @ dx
        while True:
            xn = x + t * dx
            if np.all(xn > 0) and f(xn) <= fval + alpha * t * gd:
                break
            t *= beta

        x = xn

    return x, w, hist

if __name__ == "__main__":
    n, p = 100, 30
    A, b, x0 = generate_instance(n=n, p=p, seed=20260415)

    x_star, nu_star, hist = newton_eq_entropy(A, b, x0)

    print("rank(A) =", np.linalg.matrix_rank(A))
    print("iterations =", len(hist))
```

```
    print("initial objective =", f(x0))
    print("final objective   =", f(x_star))
    print("feasibility residual =", np.linalg.norm(A @ x_star - b))
    print("stationarity residual (inf-norm) =",
          np.linalg.norm(grad(x_star) + A.T @ nu_star, ord=np.inf))

    print("\nIteration history:")
    print("k    f(x)                 lambda^2/2           ||Ax-b||_2")
    for k, fv, dec, feas in hist:
        print(f"{k:<2d}   {fv: .10f}   {dec: .10e}   {feas: .3e}")
```

■

Exercise 6:

Develop code for solving an LP in the following standard form using the barrier method:

$$\begin{array}{ll} \textbf{minimize} & c^T x \\ \text{subject to} & Ax = b,\ x \succeq 0. \end{array}$$

(a) Illustrate the progress of the algorithm in a properly chosen random instance ($n = 100, m = 500$, where $A \in \mathbb{R}^{n\times m}$); additionally, compare with CVXPY and interpret your findings.

(b) For fixed n, solve multiple instances by varying $m \in [10, 1000]$ to report the number of Newton steps needed. Compare with the expected relation (11.29) in the reference book and interpret your results.

[

$$\begin{aligned} N &\le \left\lceil \frac{\log\bigl(m/(t^{(0)}\epsilon)\bigr)}{\log\mu} \right\rceil \left(\frac{m(\mu - 1 - \log\mu)}{\gamma} + c \right) \\ &\le \left\lceil \sqrt{m}\, \frac{\log\bigl(m/(t^{(0)}\epsilon)\bigr)}{\log 2} \right\rceil \left(\frac{1}{2\gamma} + c \right) \\ &= \left\lceil \sqrt{m}\, \log_2\bigl(m/(t^{(0)}\epsilon)\bigr) \right\rceil \left(\frac{1}{2\gamma} + c \right) \\ &\le c_1 + c_2\sqrt{m}, \end{aligned} \tag{11.29}$$

where

$$c_1 = \frac{1}{2\gamma} + c, \qquad c_2 = \log_2\bigl(m/(t^{(0)}\epsilon)\bigr) \left(\frac{1}{2\gamma} + c \right).$$

]

Solution 6:
We solve

$$\begin{aligned} \textbf{minimize} \quad & c^T x \\ \text{subject to} \quad & Ax = b, \quad x \succeq 0 \end{aligned}$$

by the logarithmic barrier method. For a barrier parameter $t > 0$, consider

$$\begin{aligned} \textbf{minimize} \quad & \phi_t(x) := t\, c^T x - \sum_{i=1}^{m} \log x_i \\ \text{subject to} \quad & Ax = b. \end{aligned}$$

The gradient and Hessian are

$$\nabla \phi_t(x) = tc - \frac{1}{x}, \qquad \nabla^2 \phi_t(x) = \operatorname{diag}\left(\frac{1}{x_1^2}, \ldots, \frac{1}{x_m^2}\right).$$

Hence the equality-constrained Newton system is

$$\begin{bmatrix} \nabla^2 \phi_t(x) & A^T \\ A & 0 \end{bmatrix} \begin{bmatrix} \Delta x \\ w \end{bmatrix} = - \begin{bmatrix} \nabla \phi_t(x) \\ 0 \end{bmatrix}.$$

Since the Hessian is diagonal, we solve the reduced Schur complement system

$$AH^{-1}A^T w = -AH^{-1}g,$$

where

$$g = tc - \frac{1}{x}, \qquad H^{-1} = \operatorname{diag}(x_1^2, \ldots, x_m^2),$$

and then recover

$$\Delta x = -H^{-1}(g + A^T w).$$

A backtracking line search is used to keep $x + \alpha \Delta x \succ 0$, and the outer barrier update is

$$t \leftarrow \mu t,$$

with stopping rule

$$\frac{m}{t} < \varepsilon.$$

(a) Random instance with $n = 100$, $m = 500$. We generated a strictly feasible random instance by first sampling a full-row-rank matrix $A \in \mathbb{R}^{100 \times 500}$, then choosing $x^{(0)} \succ 0$ and setting

$$b = Ax^{(0)}.$$

Thus $x^{(0)}$ is a strictly feasible initial point. We used the parameters

$$\mu = 10, \qquad \varepsilon = 10^{-8}, \qquad \text{Newton tolerance } = 10^{-8}.$$

The barrier method produced the following summary:

runtime	$3.899693489075 \times 10^{-2}$ s
objective value	$-2.612142768173 \times 10^{2}$
total Newton steps	68
outer iterations	12
$\|Ax - b\|_2$	$1.520264944125 \times 10^{-5}$
$\min_i x_i$	$2.358572704445 \times 10^{-8}$

To illustrate the progress of the algorithm, the outer iteration history is shown below:

k	t	$c^T x^{(k)}$	m/t	Newton steps	$\|Ax^{(k)} - b\|_2$	$\min_i x_i^{(k)}$
0	10^0	1.154273×10^2	5.0×10^2	10	1.245937×10^{-11}	5.172649×10^{-1}
1	10^1	-2.228975×10^2	5.0×10^1	12	9.512006×10^{-11}	3.853261×10^{-2}
2	10^2	-2.572903×10^2	5.0×10^0	12	1.500768×10^{-9}	3.294768×10^{-3}
3	10^3	-2.608136×10^2	5.0×10^{-1}	14	1.745518×10^{-8}	3.244546×10^{-4}
4	10^4	-2.611742×10^2	5.0×10^{-2}	9	1.397381×10^{-7}	3.242947×10^{-5}
5	10^5	-2.612103×10^2	5.0×10^{-3}	7	9.703330×10^{-7}	3.252327×10^{-6}
6	10^6	-2.612139×10^2	5.0×10^{-4}	2	2.014618×10^{-6}	3.451718×10^{-7}
7	10^7	-2.612143×10^2	5.0×10^{-5}	2	1.520265×10^{-5}	2.358573×10^{-8}
8	10^8	-2.612143×10^2	5.0×10^{-6}	0	1.520265×10^{-5}	2.358573×10^{-8}
9	10^9	-2.612143×10^2	5.0×10^{-7}	0	1.520265×10^{-5}	2.358573×10^{-8}
10	10^{10}	-2.612143×10^2	5.0×10^{-8}	0	1.520265×10^{-5}	2.358573×10^{-8}
11	10^{11}	-2.612143×10^2	5.0×10^{-9}	0	1.520265×10^{-5}	2.358573×10^{-8}

We compare the result with CVXPY:

solver	status	time (s)	objective	$\|Ax - b\|_2$	obj. diff. vs barrier
`CLARABEL`	`optimal`	2.548163×10^{-1}	-2.612143×10^2	2.199587×10^{-13}	3.709935×10^{-5}
`SCS`	`optimal`	1.479883×10^{-1}	-2.612157×10^2	9.885638×10^{-6}	1.376633×10^{-3}

Interpretation. The objective value decreases rapidly and stabilizes near

$$p^\star \approx -2.612143 \times 10^2.$$

The duality-gap estimate m/t decreases by a factor of 10 at each outer iteration, exactly as expected. The barrier solution is very close to the CVXPY solution, especially to `CLARABEL`, whose objective differs only by about 3.7×10^{-5}. Thus the implementation is correct.

However, from outer iterations 8–11, the Newton step count becomes zero and the equality residual remains around 1.5×10^{-5}. This indicates numerical stagnation: once t becomes very large, the barrier subproblem becomes badly conditioned, so further increasing t no longer improves the iterate significantly. In practice, the useful convergence has already occurred by about $t = 10^6$ or 10^7.

(b) Scaling with respect to m. To study the dependence on m, we fixed $n = 5$ and varied

$$m \in \{10, 20, 50, 100, 200, 400, 600, 800, 1000\}.$$

We chose

$$\mu = 1 + \frac{1}{\sqrt{m}},$$

which is closer to the short-step schedule underlying the theoretical bound (11.29). For each m, we solved 3 random instances and recorded the total number of Newton steps. The results are:

m	μ	mean total Newton steps	std	mean outer iterations
10	1.316228	50.000000	3.265986	18.0
20	1.223607	59.666667	1.699673	28.0
50	1.141421	98.666667	0.471405	48.0
100	1.100000	153.666667	1.247219	74.0
200	1.070711	233.000000	0.000000	113.0
400	1.050000	349.000000	0.000000	171.0
600	1.040825	445.000000	0.000000	219.0
800	1.035355	527.000000	0.000000	260.0
1000	1.031623	601.000000	0.000000	297.0

A least-squares fit against $\sqrt{m}$ gives

$$N \approx 19.649522\,\sqrt{m} - 33.087274, \qquad R^2 = 0.996964.$$

A fit against $\sqrt{m}\log_2 m$ gives

$$N \approx 1.826148\,\sqrt{m}\log_2 m + 29.863497, \qquad R^2 = 0.999645.$$

Interpretation. These experiments are consistent with the theoretical relation (11.29). Indeed, the book predicts

$$N = O\big(\sqrt{m}\log(m/(t^{(0)}\epsilon))\big).$$

Over the finite range $10 \le m \le 1000$, the logarithmic factor varies slowly, so the total step count already looks nearly linear in $\sqrt{m}$. The better fit with $\sqrt{m}\log_2 m$ confirms that the observed behavior is even closer to the theoretical upper bound than a pure $\sqrt{m}$ law.

Therefore, the numerical experiment supports the conclusion that the Newton complexity of the barrier method grows roughly like $\sqrt{m}$ up to a slowly varying logarithmic factor, in agreement with the theory.

Scripts:

```
import math
import time
import numpy as np
import cvxpy as cp

np.set_printoptions(precision=6, suppress=True)


def generate_lp_feasible(n: int, m: int, seed: int = 0):
    """
    Generate a random feasible LP:
```

```
        minimize    c^T x
        subject to  A x = b, x >= 0

    We construct:
        b = A x0, with x0 > 0
    so x0 is a strictly feasible starting point.
    """
    rng = np.random.default_rng(seed)

    while True:
        A = rng.standard_normal((n, m))
        if np.linalg.matrix_rank(A) == n:
            break

    x0 = rng.uniform(0.5, 1.5, size=m)
    b = A @ x0

    # Make the problem bounded in a convenient way
    y = rng.standard_normal(n)
    s = rng.uniform(0.5, 1.5, size=m)  # strictly positive
    c = A.T @ y + s

    return A, b, c, x0


def centering_newton(A, b, c, x0, t, alpha=0.01, beta=0.5,
                     newton_tol=1e-8, max_newton=100):
    """
    Solve the equality-constrained centering problem:
        minimize    t c^T x - sum(log x_i)
        subject to  A x = b
    by Newton's method.
    """
    x = x0.astype(float).copy()
    history = []

    for it in range(max_newton + 1):
        grad = t * c - 1.0 / x
        # Hessian = diag(1/x_i^2), so inverse Hessian = diag(x_i^2)
        Hinv_diag = x ** 2

        # Schur complement system:
        # A H^{-1} A^T w = -A H^{-1} grad
        M = A @ (Hinv_diag[:, None] * A.T)
        rhs = -A @ (Hinv_diag * grad)
        w = np.linalg.solve(M, rhs)

        dx = -Hinv_diag * (grad + A.T @ w)

        lambda_sq = max(0.0, -grad @ dx)
        phi = t * (c @ x) - np.sum(np.log(x))
        eq_resid = np.linalg.norm(A @ x - b)
```

```
        history.append({
            "iter": it,
            "phi": phi,
            "lambda_sq_over_2": lambda_sq / 2.0,
            "eq_resid": eq_resid
        })

        if lambda_sq / 2.0 <= newton_tol:
            return x, it, history

        # Keep positivity
        step = 1.0
        neg_idx = dx < 0
        if np.any(neg_idx):
            step = min(step, 0.99 * np.min(-x[neg_idx] / dx[neg_idx]))

        directional_derivative = grad @ dx

        # Backtracking line search
        while True:
            x_trial = x + step * dx
            if np.all(x_trial > 0):
                phi_trial = t * (c @ x_trial) - np.sum(np.log(x_trial))
                if phi_trial <= phi + alpha * step *
                    directional_derivative:
                     break
            step *= beta
            if step < 1e-16:
                raise RuntimeError("Backtracking line search failed.")

        x = x_trial

    raise RuntimeError("Newton method did not converge.")


def barrier_method(A, b, c, x0, mu=10.0, eps=1e-8,
                   newton_tol=1e-8, max_outer=100):
    """
    Barrier method for:
        minimize    c^T x
        subject to  A x = b, x >= 0
    """
    _, m = A.shape
    x = x0.astype(float).copy()
    t = 1.0

    outer_history = []
    total_newton_steps = 0

    tic = time.time()

    for outer_it in range(max_outer):
        x, nsteps, inner_history = centering_newton(
```

```
            A=A, b=b, c=c, x0=x, t=t, newton_tol=newton_tol
        )
        total_newton_steps += nsteps

        outer_history.append({
            "outer_iter": outer_it,
            "t": t,
            "primal_obj": float(c @ x),
            "gap_estimate": float(m / t),
            "newton_steps": int(nsteps),
            "eq_residual": float(np.linalg.norm(A @ x - b)),
            "min_x": float(np.min(x)),
            "last_lambda_sq_over_2": float(inner_history[-1]["
                lambda_sq_over_2"])
        })

        if m / t < eps:
            break

        t *= mu

    toc = time.time()

    return {
        "x": x,
        "outer_history": outer_history,
        "total_newton_steps": total_newton_steps,
        "runtime_sec": toc - tic
    }


def try_cvxpy_solver(A, b, c, solver_name, x_barrier):
    x = cp.Variable(A.shape[1])
    prob = cp.Problem(cp.Minimize(c @ x), [A @ x == b, x >= 0])

    try:
        tic = time.time()
        prob.solve(solver=solver_name, verbose=False)
        toc = time.time()

        return {
            "solver": str(solver_name),
            "status": prob.status,
            "time_sec": toc - tic,
            "obj": float(prob.value),
            "eq_residual": float(np.linalg.norm(A @ x.value - b)),
            "min_x": float(np.min(x.value)),
            "rel_x_diff_vs_barrier": float(
                np.linalg.norm(x.value - x_barrier) / max(1.0, np.linalg
                    .norm(x.value))
            ),
            "obj_diff_vs_barrier": float(abs(prob.value - (c @ x_barrier
                )))
```

```
        }
    except Exception as e:
        return {
            "solver": str(solver_name),
            "status": f"FAILED: {e}"
        }


def print_dict_table(rows, columns, title=None, float_fmt="{:.6e}"):
    if title is not None:
        print("\n" + title)

    # compute widths
    widths = []
    for col in columns:
        max_len = len(col)
        for row in rows:
            val = row.get(col, "")
            if isinstance(val, float):
                s = float_fmt.format(val)
            else:
                s = str(val)
            max_len = max(max_len, len(s))
        widths.append(max_len)

    # header
    header = " | ".join(col.ljust(w) for col, w in zip(columns, widths))
    print(header)
    print("-" * len(header))

    # rows
    for row in rows:
        out = []
        for col, w in zip(columns, widths):
            val = row.get(col, "")
            if isinstance(val, float):
                s = float_fmt.format(val)
            else:
                s = str(val)
            out.append(s.ljust(w))
        print(" | ".join(out))


def run_part_a():
    print("=" * 80)
    print("PART (a): random instance with n = 100, m = 500")
    print("=" * 80)

    n, m = 100, 500
    A, b, c, x0 = generate_lp_feasible(n=n, m=m, seed=1)

    barrier = barrier_method(
        A=A, b=b, c=c, x0=x0,
```

```
        mu=10.0,
        eps=1e-8,
        newton_tol=1e-8
    )

    x_bar = barrier["x"]

    summary = {
        "runtime_sec": barrier["runtime_sec"],
        "objective": float(c @ x_bar),
        "total_newton_steps": barrier["total_newton_steps"],
        "outer_iterations": len(barrier["outer_history"]),
        "eq_residual": float(np.linalg.norm(A @ x_bar - b)),
        "min_x": float(np.min(x_bar))
    }

    print("\nBarrier summary:")
    for k, v in summary.items():
        if isinstance(v, float):
            print(f"{k:20s}: {v:.12e}")
        else:
            print(f"{k:20s}: {v}")

    print_dict_table(
        barrier["outer_history"],
        columns=[
            "outer_iter", "t", "primal_obj", "gap_estimate",
            "newton_steps", "eq_residual", "min_x"
        ],
        title="Outer iteration history",
        float_fmt="{:.6e}"
    )

    print("\nCVXPY comparison:")
    cvx_rows = []
    for solver in [cp.CLARABEL, cp.SCS]:
        cvx_rows.append(try_cvxpy_solver(A, b, c, solver, x_bar))

    # separate successful and failed rows
    success_rows = [r for r in cvx_rows if "time_sec" in r]
    fail_rows = [r for r in cvx_rows if "time_sec" not in r]

    if success_rows:
        print_dict_table(
            success_rows,
            columns=[
                "solver", "status", "time_sec", "obj", "eq_residual",
                "min_x", "rel_x_diff_vs_barrier", "obj_diff_vs_barrier"
            ],
            float_fmt="{:.6e}"
        )

    if fail_rows:
```

```
        print("\nFailed solvers:")
        for row in fail_rows:
            print(row)


def run_part_b():
    print("\n" + "=" * 80)
    print("PART (b): vary m and report total Newton steps")
    print("=" * 80)

    # To reflect the theoretical short-step style scaling,
    # we choose mu = 1 + 1/sqrt(m).
    # Fix n = 5 so that m can range from 10 to 1000.
    n_fixed = 5
    m_values = [10, 20, 50, 100, 200, 400, 600, 800, 1000]
    num_trials = 3

    rows = []

    for m in m_values:
        mu = 1.0 + 1.0 / math.sqrt(m)
        total_steps_list = []
        outer_iters_list = []

        for trial in range(num_trials):
            A, b, c, x0 = generate_lp_feasible(
                n=n_fixed, m=m, seed=11000 + 10 * m + trial
            )

            # Slightly looser tolerances here to keep the scaling
               experiment stable
            result = barrier_method(
                A=A, b=b, c=c, x0=x0,
                mu=mu,
                eps=1e-1,
                newton_tol=1e-4,
                max_outer=10000
            )

            total_steps_list.append(result["total_newton_steps"])
            outer_iters_list.append(len(result["outer_history"]))

        rows.append({
            "m": m,
            "mu": mu,
            "mean_total_newton_steps": float(np.mean(total_steps_list)),
            "std_total_newton_steps": float(np.std(total_steps_list)),
            "mean_outer_iterations": float(np.mean(outer_iters_list)),
            "sqrt_m": float(math.sqrt(m)),
            "sqrt_m_log2m": float(math.sqrt(m) * math.log2(m))
        })

    print_dict_table(
```

```
        rows,
        columns=[
            "m", "mu", "mean_total_newton_steps",
            "std_total_newton_steps", "mean_outer_iterations"
        ],
        title="Scaling results",
        float_fmt="{:.6e}"
    )

    # Fit N ~ a sqrt(m) + b
    x1 = np.array([row["sqrt_m"] for row in rows])
    y = np.array([row["mean_total_newton_steps"] for row in rows])
    coef1 = np.polyfit(x1, y, 1)
    yhat1 = np.polyval(coef1, x1)
    r2_1 = 1.0 - np.sum((y - yhat1) ** 2) / np.sum((y - np.mean(y)) **
       2)

    # Fit N ~ a sqrt(m) log2(m) + b
    x2 = np.array([row["sqrt_m_log2m"] for row in rows])
    coef2 = np.polyfit(x2, y, 1)
    yhat2 = np.polyval(coef2, x2)
    r2_2 = 1.0 - np.sum((y - yhat2) ** 2) / np.sum((y - np.mean(y)) **
       2)

    print("\nFit against sqrt(m):")
    print(f"N  {coef1[0]:.6f} * sqrt(m) + {coef1[1]:.6f}")
    print(f"R^2 = {r2_1:.6f}")

    print("\nFit against sqrt(m) * log2(m):")
    print(f"N  {coef2[0]:.6f} * sqrt(m) * log2(m) + {coef2[1]:.6f}")
    print(f"R^2 = {r2_2:.6f}")


if __name__ == "__main__":
    run_part_a()
    run_part_b()
```

■